

Report — Cosmos 3 Nano Reasoner × MMAD

⚠ Cảnh báo đọc trước: báo cáo này dựa trên **622/39,670 câu (1.57%)**, chạy trên **hai backend khác nhau** với **hai bộ trọng số khác nhau**, và **100% câu đã chạy đều thuộc một nguồn duy nhất (MVTec-AD)**. Không được dùng con số ở đây như một điểm leaderboard của Cosmos 3.

Mục 1: Thông tin cơ bản

Mục	Nội dung
Tên model test	Cosmos 3 Nano Reasoner (NVIDIA)
Dataset test trên	MMAD (ICLR 2025)
Độ phủ đã chạy	622 record / 39,670 câu = 1.57%
Ngày test	2026-07-29 → 2026-07-31
Người test	Sơn
Manifest SHA-256	ff6bf9547135bb9b58dcafcfbafb73e0cfbf804edea0b6dee57fa4dbbf5fb5d6 (giống hệt bản Qwen2-VL)

Mục 2: Setup

Chạy trên **hai backend riêng biệt**. Đây là điểm quan trọng nhất của báo cáo — hai backend cho ra kết quả khác nhau rất xa nên phải tách bạch.

Backend A — NVIDIA Build UI (endpoint chính thức)

Mục	Nội dung
Chạy ở đâu	Máy local, điều khiển trình duyệt tự động tới NVIDIA Build
Chi phí	0 đ (free tier của NVIDIA Build)
Bản model	Cosmos 3 Nano Reasoner, endpoint chính thức, không lượng tử hoá
Số record	229 (221 parse-valid)
Latency trung bình	21.0 s/câu (median 8.2 s)
Giới hạn	Mỗi request chỉ nhận 1 ảnh → bắt buộc zero-shot, không làm được 1-shot

Backend B — Kaggle T4×2 (checkpoint cộng đồng)

Mục	Nội dung
Chạy ở đâu	Kaggle, GPU T4×2 (free tier)
Chi phí	0 đ
Bản model	ThePyProgrammer/Cosmos3-Nano-reasoner-bnb8-vllm-und-only — checkpoint cộng đồng, lượng tử hoá BNB8
Nạp bằng	Qwen3VLForConditionalGeneration — vì checkpoint Cosmos khai báo kiến trúc tương thích Qwen3-VL. Trọng số vẫn là của Cosmos; KHÔNG phải thay bằng Qwen
Số record	393 (380 parse-valid)
Latency trung bình	54.7 s/câu (median 48.0 s, p95 113.4 s)
Ngân sách chạy	6.0 giờ → chỉ được 393 câu

Nhãn báo cáo bắt buộc dùng cho Backend B:

Cosmos 3 Nano Reasoner, community BNB8 quantized checkpoint, zero-shot inference on Kaggle T4×2.

Mục chung	Nội dung
Zero-shot?	CÓ — hoàn toàn zero-shot trên cả hai backend
Có train gì không?	KHÔNG . Không fine-tune, không probe, không head huấn luyện
Prompt / parser / evaluator	Giống hệt bản Qwen2-VL — cùng manifest hash, cùng system prompt, cùng parser

⚠ Hai backend không so sánh trực tiếp được với nhau: khác precision (FP gốc vs BNB8), khác runtime, khác cách sinh output. Việc gộp 622 record thành một con số duy nhất đã được đánh dấu trong `metrics_combined.json` là *"mixed-backend operational merge; not a single-model leaderboard score"*.

Mục 3: Kiểm tra leak

Giống bản Qwen2-VL: zero-shot thuần → không có tập train của mình, nên không xét cycle/participant trùng lặp hay embedding similarity train↔test.

Kiểm tra	Kết quả	Trạng thái
Cùng manifest hash với model đối chứng	ff6bf954... khớp tuyệt đối	✅ PASS
Tên file query có lộ nhãn không	Đổi thành sample_XXXX	✅ PASS
Nhãn source/class/defect trong prompt	Không có	✅ PASS
Parser có ưu ái model nào không	Cùng một parser cho cả hai model	✅ PASS
Nhiệm bản pretraining	Chưa kiểm chứng được	⚠ REVIEW
Thiên lệch độ phủ (coverage bias)	601/601 record parse-valid đều là MVTec-AD	❌ FAIL

Kết luận: không có leak theo nghĩa cổ điển, nhưng có một vấn đề **ngghiêm trọng hơn leak** về mặt hiệu lực kết quả — **thiên lệch chọn mẫu**. Cả 601 câu đã chạy đều thuộc MVTec-AD, mà theo kết quả Qwen2-VL trên bộ đầy đủ thì MVTec-AD chính là **nguồn dễ nhất** (75.25%, cao hơn MVTec-LOCO 20 điểm). Mọi con số dưới đây vì thế **lạc quan có hệ thống**, và không được dùng để so với các model đã chạy đủ 4 nguồn.

Mục 4: Kết quả chính

4.1 Theo từng backend — kèm baseline tính trên CÙNG tập record

Baseline majority-letter được tính riêng cho từng segment, trên đúng các record mà segment đó đã trả lời.

Cấu hình	n	Micro accuracy	Macro-F1	Baseline majority	Chênh lệch
Kaggle T4×2 (BNB8)	380	73.16%	0.7460	35.53%	+37.63 điểm ✓
NVIDIA Build UI (chính thức)	221	49.77%	0.4957	51.13%	−1.36 điểm ✗
Gộp cả hai (chỉ tham khảo)	601	64.56%	0.7024	41.26%	+23.30 điểm

✗ Trên endpoint chính thức của NVIDIA, Cosmos 3 Nano THUA baseline ngu — 49.77% so với 51.13% của việc luôn trả lời một chữ cái cố định.

Baseline random-choice (kỳ vọng, tính theo số lựa chọn thực của từng câu): 37.48% cho tập gộp.

Khoảng tin cậy 95% của hai segment **không chồng lấn**:

- Kaggle T4×2: 68.49% – 77.37%
- NVIDIA Build UI: 43.24% – 56.31%

Đây không phải nhiễu thống kê. Hai backend đang hành xử như hai model khác nhau.

4.2 So sánh với Qwen2-VL-2B — bắt buộc khớp nguồn

So sánh	Cosmos 3 Nano	Qwen2-VL-2B	Kết luận
✗ Sai — Cosmos (MVTec-AD) vs Qwen (toàn bộ 4 nguồn)	73.16%	64.73%	Cosmos "hơn 8.4 điểm" — so sánh không hợp lệ
✓ Đúng — cả hai trên MVTec-AD	73.16% (Kaggle, n=380)	75.25% (n=8,336)	Qwen hơn 2.1 điểm
✓ Đúng — cả hai trên MVTec-AD, gộp 2 backend	64.56% (n=601)	75.25%	Qwen hơn 10.7 điểm

Đây là điểm sửa quan trọng nhất của báo cáo. Bảng gốc trong `metrics_combined.json` đặt 73.16% cạnh 64.73% khiến Cosmos trông vượt trội; thực tế khi khớp nguồn thì Cosmos **không** vượt Qwen2-VL-2B.

4.3 Theo task (segment Kaggle T4×2, n = 380)

Task	n	Accuracy
Object Classification	23	100.00%
Object Analysis	67	94.03%
Defect Analysis	53	84.91%
Defect Localization	56	82.14%
Defect Description	54	75.93%
Defect Classification	48	60.42%
Anomaly Detection	79	39.24%

4.4 Anomaly Detection — dữ liệu mất cân bằng, không dùng accuracy

Trong 79 câu AD của segment Kaggle, 56 câu (70.89%) có nhãn "có lỗi".

	Precision	Recall	F1	Miss rate	Overkill
Cosmos 3 Nano (BNB8)	1.0000	0.1429	0.2500	85.71%	0.00%
Baseline "luôn nói có lỗi"	0.7089	1.0000	0.8295	0%	100%
Chênh lệch	+0.2911	-0.8571	-0.5795		

✖

THUA baseline rất nặng. Cosmos đúng tuyệt đối khi nó dám khẳng định có lỗi (precision 1.0 — không báo động giả lần nào), nhưng nó **chỉ dám khẳng định 8/56 lần**. 48/56 ca lỗi thật bị bỏ qua.

Đây là hành vi "luôn nói không có lỗi" — an toàn cho chỉ số precision nhưng **vô dụng cho một cổng kiểm tra chất lượng công nghiệp**.

4.5 Tỷ lệ hoàn thành và độ trễ

	Kaggle T4×2	NVIDIA Build UI
Record thử	393	229
Parse-valid	380 (96.7%)	221 (96.5%)
Parse failure	13 (3.31%)	8
Latency mean	54.7 s	21.0 s
Latency median	48.0 s	8.2 s
Completion rate so với 39,670 câu	0.996%	0.58%

Với 54.7 s/câu, chạy hết phần còn lại cần **≈ 600 giờ GPU**. Đây là lý do cơ chế shared checkpoint qua GitHub được xây (đã code xong, chưa seed shard nào).

Mục 5: Kiểm tra shortcut

MMAD là ảnh tĩnh đơn → **test static-frame không áp dụng**.

Kiểm tra	Trạng thái
Blind-image control (thay ảnh, giữ câu hỏi)	✗ CHƯA LÀM
Answer-position bias	✗ CHƯA ĐO (mẫu 601 quá nhỏ để đo tin cậy)

Có một quan sát gián tiếp đáng chú ý từ Mục 6: reasoning của Cosmos trên endpoint chính thức **mô tả ảnh khá chi tiết và chính xác** (màu, hình, bề mặt, ánh sáng) rồi mới kết luận sai. Điều này gợi ý model **có nhìn ảnh** — lỗi nằm ở bước phán đoán bất thường chứ không phải ở bước tri giác. Nhưng đây là quan sát định tính, không thay thế được blind-image control.

Mục 6: Ví dụ lỗi cụ thể

Ưu điểm của Cosmos so với Qwen: nó xuất ra chuỗi suy luận `<think>...</think>` nên truy được nguyên nhân sai.

Lỗi 1 — `mmad_full_00004` (MVTec-AD / bottle / Anomaly Detection / NVIDIA Build UI)

- Câu hỏi:** "Is there any defect in the object?" — A) Yes. B) No.
- Đáp án thật:** A — "Yes." (ảnh có lỗi)
- Model đoán:** B — "No."
- Suy luận của model:** "...the object has a smooth, polished surface with subtle reflections and highlights. It's uniformly black with no visible scratches, dents, or irregularities..."
- Loại lỗi: Thiếu thông tin đầu vào / mất chi tiết do xử lý.** Model mô tả bề mặt đúng nhưng khẳng định "không có bất thường nào nhìn thấy được" — lỗi trên MVTec-AD bottle thường rất nhỏ, có khả năng đã bị mất khi ảnh qua pipeline nén/resize của UI.

Lỗi 2 — `mmad_full_00008` (MVTec-AD / bottle / Anomaly Detection / NVIDIA Build UI)

- Đáp án thật:** A — "Yes." | **Model đoán:** B — "No."
- Suy luận của model:** "...the edges are slightly blurred, which might be due to the camera's depth of fi[eld]..."
- Loại lỗi: Model hallucinate — bịa lý do để bỏ qua bằng chứng.** Model có thấy điểm bất thường (cạnh mờ) nhưng tự nghĩ ra một lời giải thích lành tính ("do độ sâu trường ảnh") rồi loại nó khỏi kết luận. Đây là dạng lỗi nguy hiểm nhất trong kiểm tra chất lượng: model hợp lý hoá khuyết tật thành hiện tượng quang học.

Lỗi 3 — `mmad_full_00021` (MVTec-AD / bottle / Anomaly Detection / NVIDIA Build UI)

- Câu hỏi:** A) No. B) Yes. (thứ tự đảo so với Lỗi 1)
- Đáp án thật:** B — "Yes." | **Model đoán:** A — "No."
- Suy luận:** "...the object has concentric ridges... metallic sheen... slight reflective highlight on one side..."
- Loại lỗi: Lý do khác — thiên lệch hệ thống về phía "không có lỗi".** Ba ca liên tiếp đều sai theo cùng một chiều, kể cả khi thứ tự đáp án bị đảo → không phải thiên lệch vị trí, mà là thiên lệch nội dung. Khớp với miss rate 85.71%.

Lỗi 4 — `mmad_full_08429` (MVTec-AD / cable / Defect Classification / Kaggle T4×2)

- Câu hỏi:** "What is the type of the defect?" — A) Poke insulation. B) Fraying. C) Abrasion. D) Discoloration.

- **Đáp án thật:** A — "Poke insulation." | **Model đoán:** C — "Abrasion."
- **Suy luận:** model liệt kê đúng định nghĩa từng loại ("*Poke insulation usually refers to the insulation being punctured... Fraying would be...*") rồi vẫn chọn sai.
- **Loại lỗi: Nhầm lẫn giữa 2 nhãn quá giống nhau.** Model nắm đúng khái niệm nhưng không ánh xạ được vào bằng chứng thị giác cụ thể — kiến thức có, grounding thiếu.

Lỗi 5 — `mmad_full_08433` (MVTec-AD / capsule / Defect Classification / Kaggle T4×2)

- **Câu hỏi:** A) Crack. B) Leakage. C) Discoloration. D) Dent.
- **Đáp án thật:** A — "Crack." | **Model đoán:** C — "Discoloration."
- **Suy luận:** "*...the orange section has a slight discoloration or uneven texture. Also, there's a sma[ll]...*" — model nhận ra cả hai manh mối nhưng chọn manh mối nổi bật hơn về màu sắc.
- **Loại lỗi: Lý do khác — chọn tín hiệu nổi bật thay vì tín hiệu đúng.** Vết nứt nhỏ khó thấy hơn vùng đổi màu, model bám vào cái dễ thấy.

Tổng hợp phân bố lỗi (601 record parse-valid, 213 sai)

Task	Số lỗi	% tổng lỗi
Anomaly Detection	159	74.6%
Defect Classification	19	8.9%
Defect Description	13	6.1%
Defect Localization	10	4.7%
Defect Analysis	8	3.8%
Object Analysis	4	1.9%

Gần 3/4 toàn bộ lỗi tập trung vào đúng một task — Anomaly Detection. Theo backend: NVIDIA Build UI 111 lỗi / Kaggle T4×2 102 lỗi.

Mục 7: Kết luận cuối cùng

Vì hai backend chạy hai bộ trọng số khác nhau và cho kết quả có khoảng tin cậy không chồng lấn, **một kết luận chung ở mức dataset là không bảo vệ được.** Ghi riêng theo từng cấu hình:

Cosmos 3 Nano — BNB8 cộng đồng, Kaggle T4×2 — trên MMAD (subset MVTec-AD, n = 380)

- ☒ **THẮNG RÕ RÀNG** so với baseline đơn giản — 73.16% vs 35.53%
- ☐ NGANG BẰNG
- ☐ THUA

Cosmos 3 Nano — endpoint chính thức NVIDIA Build — trên MMAD (subset MVTec-AD, n = 221)

- ☐ THẮNG RÕ RÀNG
- ☐ NGANG BẰNG
- ☒ **THUA** baseline đơn giản — 49.77% vs 51.13%

Kết luận phụ — subtask Anomaly Detection (n = 79, segment Kaggle)

- ☒ **THUA** baseline đơn giản — F1 0.250 vs 0.830

Kết luận so với Qwen2-VL-2B (khớp nguồn MVTec-AD): Cosmos 3 Nano **chưa vượt** Qwen2-VL-2B — 73.16% (bản BNB8, cấu hình tốt nhất) so với 75.25%. Trong khi đó Cosmos chậm hơn **79x** (54.7 s vs 0.69 s mỗi câu).

Mục 8: Lưu ý / giới hạn

Mẫu test có đủ lớn không?

KHÔNG. $622/39,670 = 1.57\%$. Nghiêm trọng hơn: 601/601 record parse-valid đều thuộc **một nguồn duy nhất** (MVTec-AD), không có câu nào của GoodsAD, VisA hay MVTec-LOCO. Một số subtask chỉ có 23 câu (Object Classification). Đây chưa đủ để kết luận về năng lực của Cosmos 3 trên MMAD.

Có phát hiện leak không? Kết quả Mục 4 có đáng tin?

Không có leak protocol. Nhưng có **thiên lệch độ phủ nghiêm trọng** làm kết quả lạc quan có hệ thống — MVTec-AD là nguồn dễ nhất (theo Qwen: 75.25% vs 55.22% của MVTec-LOCO). Nếu chạy đủ 4 nguồn, con số 73.16% gần như chắc chắn sẽ giảm. Vì vậy Mục 4 **chỉ đáng tin trong phạm vi MVTec-AD**, và mọi so sánh liên model bắt buộc phải khớp nguồn.

Có điều gì chưa kiểm tra mà làm thêm sẽ chắc chắn hơn?

1. **Giải thích chênh lệch 73.16% vs 49.77% giữa hai backend** — đây là việc cấp bách nhất. Cho tới khi hiểu nguyên nhân (lượng tử hoá BNB8? khác nhau ở xử lý ảnh phía UI? khác prompt template?), mọi record gộp về sau đều mang lỗi hệ thống này. Cách kiểm: chạy **cùng 100 câu giống hệt nhau** trên cả hai backend rồi so từng câu một.
 2. **Mở rộng sang 3 nguồn còn lại** — ưu tiên MVTec-LOCO (nguồn khó nhất) để có cận dưới thực tế.
 3. **Blind-image control** — chưa làm.
 4. **Chưa lưu probability outputs** → không claim được calibration hay abstention, dù precision 1.0 gợi ý model đang tự abstain ngầm.
 5. **Chưa seed shared checkpoint** — code đã xong (`common/shared_checkpoint.py` , 3 test pass) nhưng thư mục `checkpoints/cosmos3_mmad/` mới chỉ có README + manifest, chưa có shard dữ liệu nào. Đây là nút thắt để chạy tiếp.
 6. **Chưa đối chiếu với GPT-4o ~74.9%** một cách hợp lệ — mốc đó tính trên toàn bộ MMAD, không phải riêng MVTec-AD.
-

Nguồn dữ liệu và mã nguồn

Loại	Đường dẫn
Prediction gộp	<code>research/mmad_model_benchmark/outputs/cosmos3_combined/predictions_combined.jsonl / .csv</code>
Metrics gộp	<code>.../outputs/cosmos3_combined/metrics_combined.json</code>
Phân tích theo segment	<code>.../outputs/cosmos3_combined/analysis_summary.json</code> , <code>analysis_by_segment.csv</code>
Metrics Kaggle T4x2	<code>.../outputs/cosmos3_t4x2/metrics.json</code>
Notebook Kaggle	<code>.../models/cosmos3/Cosmos3_Nano_MMAD_T4x2_UI_Parity.ipynb</code>
Runner NVIDIA Build	<code>.../models/cosmos3/run_nvidia_build.py</code>
Notebook phân tích	<code>.../analysis/cosmos3_mmad_combined_analysis.ipynb</code> (+ bản <code>.html</code>)
Repo public	<code>https://github.com/anhsohn/mini-world-model</code>